

# C++ Roadmap — A to Z

25 topics covering C++ from basics to memory, STL & WinAPI for RE

*Generated by BlasterX24 — learn · practice · test*

# Table of Contents

|                                              |
|----------------------------------------------|
| Topic 01 — Setup & First Program             |
| Topic 02 — Variables & Types                 |
| Topic 03 — Operators & Expressions           |
| Topic 04 — Control Flow                      |
| Topic 05 — Functions                         |
| Topic 06 — Arrays & C-Strings                |
| Topic 07 — Pointers & References             |
| Topic 08 — Dynamic Memory                    |
| Topic 09 — Strings (std::string)             |
| Topic 10 — Structs & Enums                   |
| Topic 11 — OOP: Classes & Objects            |
| Topic 12 — OOP: Inheritance & Polymorphism   |
| Topic 13 — Operator Overloading              |
| Topic 14 — Templates                         |
| Topic 15 — STL: Containers                   |
| Topic 16 — STL: Iterators & Algorithms       |
| Topic 17 — Lambda & Modern C++               |
| Topic 18 — Smart Pointers & RAII             |
| Topic 19 — Exception Handling                |
| Topic 20 — File I/O                          |
| Topic 21 — Preprocessor                      |
| Topic 22 — Memory Layout & Internals         |
| Topic 23 — Bitwise Operations & Manipulation |
| Topic 24 — WinAPI Basics for RE              |
| Topic 25 — Capstone Projects                 |

# Topic 1: Setup & First Program

---

C++ is a compiled language requiring a compiler. g++ is the common free compiler. Every program starts with main(). Use #include for headers and std::cout for output.

## Learn From

Beginner tutorials for C++ setup.

- GCC — <https://gcc.gnu.org/>
- C++ Reference — <https://cplusplus.com/reference/iostream/>
- LearnCpp — <https://www.learncpp.com/>

## Key Concepts

- g++ compiler
- #include <iostream>
- int main()
- std::cout <<
- std::endl
- Compiling with g++
- Return values from main()

### Problem 1

Print 'Hello, World!'.

```
#include <iostream>
int main(){std::cout<<"Hello, World!"<<std::endl;return 0;}
```

*Expected output:*

Hello, World!

*Learning points:*

### Problem 2

Print your name.

```
#include <iostream>
int main(){std::cout<<"BlasterX24"<<std::endl;return 0;}
```

*Expected output:*

BlasterX24

*Learning points:*

### Problem 3

Print two lines.

```
#include <iostream>
int main(){std::cout<<"Line 1"<<std::endl;std::cout<<"Line 2"<<std::endl;return 0;}
```

*Expected output:*

Line 1

Line 2

*Learning points:*

### Problem 4

Print 7+3.

```
#include <iostream>
int main(){std::cout<<7+3<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

### Problem 5

Print A B C on separate lines.

```
#include <iostream>
int main(){std::cout<<'A'<<std::endl;std::cout<<'B'<<std::endl;std::cout<<'C'<<std::endl;return 0;}
```

*Expected output:*

A

B

C

*Learning points:*

### Problem 6

Return 42 from main().

```
#include <iostream>
int main(){return 42;}
```

*Learning points:*

### Problem 7

Print 'C++ C++ C++'.

```
#include <iostream>
int main(){std::cout<<"C++ C++ C++"<<std::endl;return 0;}
```

*Expected output:*

C++ C++ C++

*Learning points:*

## Problem 8

Print character 'x'.

```
#include <iostream>
int main(){std::cout<<'x'<<std::endl;return 0;}
```

*Expected output:*

x

*Learning points:*

## Problem 9

Print 'Start, End'.

```
#include <iostream>
int main(){std::cout<<"Start, End"<<std::endl;return 0;}
```

*Expected output:*

Start, End

*Learning points:*

## Problem 10

Print 3.14.

```
#include <iostream>
int main(){std::cout<<3.14<<std::endl;return 0;}
```

*Expected output:*

3.14

*Learning points:*

## Quick Questions & Answers

**Q1: Compiler command for main.cpp to app?**

A: g++ main.cpp -o app

**Q2: Header for std::cout?**

A: #include <iostream>

**Q3: Return type of main()?**

A: int

**Q4: What does std::endl do?**

A: Prints newline and flushes buffer.

**Q5: Entry point of C++ program?**

A: main() function.

**Q6: Multiple main() allowed?**

A: No, only one.

**Q7: return 0 signifies?**

A: Successful execution.

**Q8: \n vs std::endl?**

A: `\n` is newline only; `endl` flushes too.

**Q9: `cout` namespace?**

A: `std` namespace.

**Q10: C++17 flag?**

A: `-std=c++17`

## Topic 2: Variables & Types

---

C++ is statically typed. Fundamental types include int, char, float, double, bool. sizeof returns byte count. Each type has defined ranges.

### Learn From

Fundamental types references.

- C++ Variables — <https://cplusplus.com/doc/tutorial/variables/>
- cppreference sizeof — <https://en.cppreference.com/w/cpp/language/sizeof>

### Key Concepts

- int, char, float, double, bool
- sizeof operator
- Signed vs unsigned
- Type ranges
- Declaration and initialization
- const variables
- auto deduction

### Problem 1

Declare int x=10 and print.

```
#include <iostream>
int main(){int x=10;std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

### Problem 2

Print sizeof(int).

```
#include <iostream>
int main(){std::cout<<sizeof(int)<<std::endl;return 0;}
```

*Expected output:*

4

*Learning points:*

### Problem 3

Print sizeof(double).

```
#include <iostream>
int main(){std::cout<<sizeof(double)<<std::endl;return 0;}
```

*Expected output:*

8

*Learning points:*

#### Problem 4

Declare char c='Z' and print.

```
#include <iostream>
int main(){char c='Z';std::cout<<c<<std::endl;return 0;}
```

*Expected output:*

Z

*Learning points:*

#### Problem 5

Declare bool b=true and print.

```
#include <iostream>
int main(){bool b=true;std::cout<<b<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

#### Problem 6

Print sizeof(char).

```
#include <iostream>
int main(){std::cout<<sizeof(char)<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

#### Problem 7

Declare float f=2.5f and print.

```
#include <iostream>
int main(){float f=2.5f;std::cout<<f<<std::endl;return 0;}
```

*Expected output:*

2.5

*Learning points:*

#### Problem 8

Use auto for y=42.

```
#include <iostream>
int main(){auto y=42;std::cout<<y<<std::endl;return 0;}
```

*Expected output:*

42

*Learning points:*



## Problem 9

Declare const int MAX=100 and print.

```
#include <iostream>
int main(){const int MAX=100;std::cout<<MAX<<std::endl;return 0;}
```

*Expected output:*

100

*Learning points:*

## Problem 10

Print sizeof(long long).

```
#include <iostream>
int main(){std::cout<<sizeof(long long)<<std::endl;return 0;}
```

*Expected output:*

8

*Learning points:*

## Quick Questions & Answers

### Q1: sizeof int?

A: 4 bytes.

### Q2: signed vs unsigned?

A: Signed holds negatives; unsigned larger positive range.

### Q3: signed char range?

A: -128 to 127.

### Q4: auto keyword?

A: Deduces type from initializer.

### Q5: const purpose?

A: Makes variable read-only.

### Q6: unsigned int max 32-bit?

A: 4294967295.

### Q7: float literal to double?

A: Yes, implicit conversion.

### Q8: 3.14 vs 3.14f?

A: 3.14 is double; 3.14f is float.

### Q9: INT\_MIN/INT\_MAX header?

A: <climits>.

### Q10: Overflow signed int?

A: Undefined behavior.

## Topic 3: Operators & Expressions

---

Operators perform operations on operands. Includes arithmetic (+, -, \*, /, %), comparison (==, !=, <, >), logical (&&, ||, !), bitwise (&, |, ^, ~, <<, >>), and ternary (? :).

### Learn From

Operators references.

- C++ Operators — <https://cplusplus.com/doc/tutorial/operators/>
- cppreference Operators — <https://en.cppreference.com/w/cpp/language/operator>

### Key Concepts

- Arithmetic operators
- Comparison operators
- Logical operators
- Bitwise operators
- Ternary operator
- Operator precedence
- Type promotion

### Problem 1

Print 15/4 integer division.

```
#include <iostream>
int main(){std::cout<<15/4<<std::endl;return 0;}
```

*Expected output:*

3

*Learning points:*

### Problem 2

Print 17%5.

```
#include <iostream>
int main(){std::cout<<17%5<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

### Problem 3

Print (5>3).

```
#include <iostream>
int main(){std::cout<<(5>3)<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

## Problem 4

Print  $3+4*2$ .

```
#include <iostream>
int main(){std::cout<<3+4*2<<std::endl;return 0;}
```

*Expected output:*

11

*Learning points:*

## Problem 5

Ternary max of 10,20.

```
#include <iostream>
int main(){int a=10,b=20;std::cout<<(a>b?a:b)<<std::endl;return 0;}
```

*Expected output:*

20

*Learning points:*

## Problem 6

Print  $6\&3$ .

```
#include <iostream>
int main(){std::cout<<(6&3)<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

## Problem 7

Print  $1<<3$ .

```
#include <iostream>
int main(){std::cout<<(1<<3)<<std::endl;return 0;}
```

*Expected output:*

8

*Learning points:*

## Problem 8

Print  $(5>3\&\&10>7)$ .

```
#include <iostream>
int main(){std::cout<<(5>3&\&10>7)<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

## Problem 9

Print 10/3.0.

```
#include <iostream>
int main(){std::cout<<10/3.0<<std::endl;return 0;}
```

*Expected output:*

3.33333

*Learning points:*

## Problem 10

Print 5^3.

```
#include <iostream>
int main(){std::cout<<(5^3)<<std::endl;return 0;}
```

*Expected output:*

6

*Learning points:*

## Quick Questions & Answers

### Q1: Operator precedence?

A: Rules for evaluation order.

### Q2: 10/3 in C++?

A: 3 (truncates).

### Q3: % operator?

A: Remainder of division.

### Q4: Ternary operator?

A: condition?a:b.

### Q5: Type promotion?

A: Implicit smaller to larger type.

### Q6: << bitwise?

A: Left shift.

### Q7: !0 result?

A: 1 (true).

### Q8: Short-circuit evaluation?

A: Right not evaluated if left determines result.

### Q9: ~ operator?

A: Bitwise NOT.

### Q10: == vs =?

A: == compares; = assigns.

## Topic 4: Control Flow

---

Control flow determines execution order. Conditionals (if/else, switch) make decisions; loops (for, while, do-while) repeat code. break exits; continue skips iteration.

### Learn From

Control flow references.

- C++ Control — <https://cplusplus.com/doc/tutorial/control/>

### Key Concepts

- if/else if/else
- switch-case with default
- for loop
- while loop
- do-while loop
- break and continue
- Nested control

### Problem 1

Print Even if 4 is even.

```
#include <iostream>
int main(){int n=4;if(n%2==0)std::cout<<"Even"<<std::endl;else std::cout<<"Odd"<<std::endl;return 0;}
```

*Expected output:*

Even

*Learning points:*

### Problem 2

For loop 1 to 5.

```
#include <iostream>
int main(){for(int i=1;i<=5;i++)std::cout<<i<<std::endl;return 0;}
```

*Expected output:*

1  
2  
3  
4  
5

*Learning points:*

### Problem 3

Print first 5 even numbers.

```
#include <iostream>
int main(){for(int i=1;i<=5;i++)std::cout<<i*2<<std::endl;return 0;}
```

*Expected output:*

2  
4  
6  
8  
10

*Learning points:*

## Problem 4

While countdown 5 to 1.

```
#include <iostream>
int main(){int i=5;while(i>=1){std::cout<<i<<std::endl;i--;}return 0;}
```

*Expected output:*

5  
4  
3  
2  
1

*Learning points:*

## Problem 5

Switch day=3 (Monday=1).

```
#include <iostream>
int main(){int day=3;switch(day){case 1:std::cout<<"Monday"<<std::endl;break;case 2:std::cout<<"Tuesday"<<std::endl;break;case 3:std::cout<<"Wednesday"<<std::endl;break;default:std::cout<<"Invalid day"<<std::endl;}return 0;}
```

*Expected output:*

Wednesday

*Learning points:*

## Problem 6

Break when i=3 (1-5).

```
#include <iostream>
int main(){for(int i=1;i<=5;i++){if(i==3)break;std::cout<<i<<std::endl;}return 0;}
```

*Expected output:*

1  
2

*Learning points:*

## Problem 7

Continue skip 3 (1-5).

```
#include <iostream>
int main(){for(int i=1;i<=5;i++){if(i==3)continue;std::cout<<i<<std::endl;}return 0;}
```

*Expected output:*

1  
2  
4  
5

*Learning points:*

### Problem 8

Do-while 1 to 3.

```
#include <iostream>
int main(){int i=1;do{std::cout<<i<<std::endl;i++;}while(i<=3);return 0;}
```

*Expected output:*

1  
2  
3

*Learning points:*

### Problem 9

Print Positive if 7>0.

```
#include <iostream>
int main(){int n=7;if(n>0)std::cout<<"Positive"<<std::endl;return 0;}
```

*Expected output:*

Positive

*Learning points:*

### Problem 10

Sum 1 to 5 with for loop.

```
#include <iostream>
int main(){int sum=0;for(int i=1;i<=5;i++)sum+=i;std::cout<<sum<<std::endl;return 0;}
```

*Expected output:*

15

*Learning points:*

## Quick Questions & Answers

#### Q1: while vs do-while?

A: while checks first; do-while executes at least once.

#### Q2: break in switch?

A: Exits switch block.

#### Q3: continue in loop?

A: Skips current iteration.

#### Q4: switch on strings?

A: No, only integral types.

**Q5: default case?**

A: Handles unmatched values.

**Q6: Missing break in case?**

A: Falls through to next case.

**Q7: Nested for loops?**

A: Yes, arbitrary depth.

**Q8: Loop variable scope?**

A: Local to for loop.

**Q9: When use do-while?**

A: When body must run at least once.

**Q10: Float loop counter?**

A: Yes but not recommended.



## Topic 5: Functions

---

Functions are reusable code blocks. C++ supports overloading, pass by value/reference, and inline hint. Functions organize code and reduce duplication.

### Learn From

Function references.

- C++ Functions — <https://cplusplus.com/doc/tutorial/functions/>

### Key Concepts

- Declaration vs definition
- Parameters vs arguments
- Return types
- Pass by value vs reference
- Function overloading
- Inline functions
- Default parameters
- Variable scope

### Problem 1

Function double int value.

```
#include <iostream>
double doubleValue(int x){return x*2.0;}
int main(){std::cout<<doubleValue(5)<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

### Problem 2

Function sum two ints.

```
#include <iostream>
int add(int a,int b){return a+b;}
int main(){std::cout<<add(3,7)<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

### Problem 3

Function double by reference.

```
#include <iostream>
void doubleRef(int &x){x*=2;}
int main(){int a=5;doubleRef(a);std::cout<<a<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

## Problem 4

Overloaded max for int and double.

```
#include <iostream>
int max(int a,int b){return a>b?a:b;}
double max(double a,double b){return a>b?a:b;}
int main(){std::cout<<max(3,7)<<std::endl;std::cout<<max(2.5,1.5)<<std::endl;return 0;}
```

*Expected output:*

7

2.5

*Learning points:*

## Problem 5

Default parameter greeting.

```
#include <iostream>
#include <string>
void greet(std::string name="World"){std::cout<<"Hello, "<<name<<std::endl;}
int main(){greet();return 0;}
```

*Expected output:*

Hello, World

*Learning points:*

## Problem 6

Function return larger of two.

```
#include <iostream>
int maximum(int a,int b){return (a>b)?a:b;}
int main(){std::cout<<maximum(10,20)<<std::endl;return 0;}
```

*Expected output:*

20

*Learning points:*

## Problem 7

Function sum array.

```
#include <iostream>
int sumArray(int arr[],int size){int s=0;for(int i=0;i<size;i++)s+=arr[i];return s;}
int main(){int arr[]={1,2,3};std::cout<<sumArray(arr,3)<<std::endl;return 0;}
```

*Expected output:*

6

*Learning points:*

## Problem 8

Function swap by reference.

```
#include <iostream>
void swapVals(int &a,int &b){int temp=a;a=b;b=temp;}
int main(){int x=1,y=2;swapVals(x,y);std::cout<<x<<" "<<y<<std::endl;return 0;}
```

*Expected output:*

2 1

*Learning points:*

## Problem 9

Inline square function.

```
#include <iostream>
inline int square(int x){return x*x;}
int main(){std::cout<<square(4)<<std::endl;return 0;}
```

*Expected output:*

16

*Learning points:*

## Problem 10

Recursive factorial(5).

```
#include <iostream>
int factorial(int n){if(n<=1)return 1;return n*factorial(n-1);}
int main(){std::cout<<factorial(5)<<std::endl;return 0;}
```

*Expected output:*

120

*Learning points:*

## Quick Questions & Answers

### Q1: Declaration vs definition?

A: Declaration tells signature; definition provides code.

### Q2: Pass by reference?

A: Pass address to modify original.

### Q3: Function overloading?

A: Same name, different parameter lists.

### Q4: Inline suggestion?

A: Insert body at call site.

### Q5: Return reference?

A: Yes but not to local.

### Q6: Default parameters?

A: Take default if no arg.

**Q7: Non-void no return?**

A: Undefined behavior.

**Q8: Recursion?**

A: Function calling itself with base case.

**Q9: Function pointer?**

A: Stores function address.

**Q10: Overload by return type?**

A: No, params must differ.

## Topic 6: Arrays & C-Strings

---

Arrays hold same-type elements contiguously. C-strings are char arrays terminated by null. <cstring> has strlen, strcpy, strcmp, strcat. Bounds not checked.

### Learn From

Array references.

- C++ Arrays — <https://cplusplus.com/doc/tutorial/arrays/>
- cstring — <https://en.cppreference.com/w/cpp/header/cstring>

### Key Concepts

- Array declaration
- Array size with sizeof
- C-strings null termination
- strlen, strcpy, strcmp, strcat
- Bounds undefined behavior
- Multi-dimensional arrays
- Passing arrays to functions

### Problem 1

Print array {1,2,3,4,5}.

```
#include <iostream>
int main(){int arr[]={1,2,3,4,5};for(int i=0;i<5;i++)std::cout<<arr[i]<<std::endl;return 0;}
```

*Expected output:*

```
1
2
3
4
5
```

*Learning points:*

### Problem 2

Print array size.

```
#include <iostream>
int main(){int arr[]={10,20,30,40,50};std::cout<<sizeof(arr)/sizeof(arr[0])<<std::endl;return 0;}
```

*Expected output:*

```
5
```

*Learning points:*

### Problem 3

Print strlen of Hello.

```
#include <iostream>
#include <cstring>
int main(){char str[]="Hello";std::cout<<strlen(str)<<std::endl;return 0;}
```

*Expected output:*

5

*Learning points:*

## Problem 4

Copy string.

```
#include <iostream>
#include <cstring>
int main(){char dest[20];strcpy(dest,"World");std::cout<<dest<<std::endl;return 0;}
```

*Expected output:*

World

*Learning points:*

## Problem 5

Compare strings.

```
#include <iostream>
#include <cstring>
int main(){std::cout<<strcmp("abc","abd")<<std::endl;return 0;}
```

*Expected output:*

-1

*Learning points:*

## Problem 6

Find max in array.

```
#include <iostream>
int main(){int arr[]={3,7,2,9,4};int max=arr[0];for(int i=1;i<5;i++)if(arr[i]>max)max=arr[i];std::cout<<max<<std::endl;return 0;}
```

*Expected output:*

9

*Learning points:*

## Problem 7

Reverse array.

```
#include <iostream>
int main(){int arr[]={1,2,3,4,5};for(int i=4;i>=0;i--)std::cout<<arr[i]<<({i>0?" ":" "});std::cout<<std::endl;return 0;}
```

*Expected output:*

5 4 3 2 1

*Learning points:*

## Problem 8

Concatenate strings.

```
#include <iostream>
#include <cstring>
int main(){char str[20]="Hello";strcat(str," there");std::cout<<str<<std::endl;return 0;}
```

*Expected output:*

Hello there

*Learning points:*

## Problem 9

Count vowels in Hello World.

```
#include <iostream>
#include <cstring>
int main(){char str[]="Hello World";int c=0;for(int i=0;str[i];i++){char ch=str[i];if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')c++;}std::cout<<c<<std::endl;return 0;}
```

*Expected output:*

3

*Learning points:*

## Problem 10

2D array print.

```
#include <iostream>
int main(){int arr[2][3]={{1,2,3},{4,5,6}};for(int i=0;i<2;i++)for(int j=0;j<3;j++)std::cout<<arr[i][j]<<std::endl;}
```

*Expected output:*

1  
2  
3  
4  
5  
6

*Learning points:*

## Quick Questions & Answers

### Q1: C-string vs char array?

A: C-string terminated by null.

### Q2: Bounds danger?

A: Undefined behavior.

### Q3: strlen empty string?

A: 0.

### Q4: strcpy overflow?

A: Buffer overflow.

### Q5: Array size?

A: sizeof(arr)/sizeof(arr[0]).

**Q6: Assign arrays with =?**

A: No, must copy.

**Q7: strcmp equal?**

A: 0.

**Q8: 2D array storage?**

A: Row-major order.

**Q9: Null terminator?**

A: '\0' marks string end.

**Q10: arr vs &arr?**

A: arr decays to pointer; &arr is full array.



## Topic 7: Pointers & References

---

Pointers store memory addresses. \* dereferences; & takes address. Pointer arithmetic traverses arrays. nullptr is modern null. void\* is generic pointer.

### Learn From

Pointer references.

- C++ Pointers — <https://cplusplus.com/doc/tutorial/pointers/>

### Key Concepts

- Pointer declaration
- Dereferencing \*
- Address-of &
- Pointer arithmetic
- nullptr
- void\* pointer
- const pointers
- References vs pointers

### Problem 1

Dereference pointer to print.

```
#include <iostream>
int main(){int x=10;int *p=&x;std::cout<<*p<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

### Problem 2

Pointer modifies x 5 to 20.

```
#include <iostream>
int main(){int x=5;int *p=&x;*p=20;std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

20

*Learning points:*

### Problem 3

Print address of variable.

```
#include <iostream>
int main(){int x=42;std::cout<<&x<<std::endl;return 0;}
```

*Expected output:*

0x

*Learning points:*

## Problem 4

Pointer arithmetic traverse array.

```
#include <iostream>
int main(){int arr[]={10,20,30};int *p=arr;for(int i=0;i<3;i++)std::cout<<*(p+i)<<std::endl;return 0;}
```

*Expected output:*

10  
20  
30

*Learning points:*

## Problem 5

nullptr check.

```
#include <iostream>
int main(){int *p=nullptr;if(p==nullptr)std::cout<<"null"<<std::endl;return 0;}
```

*Expected output:*

null

*Learning points:*

## Problem 6

Reference modifies variable.

```
#include <iostream>
int main(){int x=5;int &ref=x;ref=15;std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

15

*Learning points:*

## Problem 7

Const pointer.

```
#include <iostream>
int main(){int x=10;int *const p=&x;std::cout<<*p<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

## Problem 8

Void\* points to double.

```
#include <iostream>
int main(){double d=3.14;void *vp=&d;std::cout<<*(double*)vp<<std::endl;return 0;}
```

*Expected output:*

3.14

*Learning points:*

## Problem 9

Pointer difference.

```
#include <iostream>
int main(){int arr[]={10,20,30,40};int *p1=&arr[0];int *p2=&arr[3];std::cout<<p2-p1<<std::endl;return 0;}
```

*Expected output:*

3

*Learning points:*

## Problem 10

Swap with pointers.

```
#include <iostream>
void swapPtr(int *a,int *b){int temp=*a;*a=*b;*b=temp;}
int main(){int x=1,y=2;swapPtr(&x,&y);std::cout<<x<<" "<<y<<std::endl;return 0;}
```

*Expected output:*

2 1

*Learning points:*

## Quick Questions & Answers

### Q1: Pointer?

A: Stores memory address.

### Q2: \* in declaration?

A: Declares pointer type.

### Q3: Pointer vs reference?

A: Pointer can be null/reassigned; reference cannot.

### Q4: nullptr?

A: Modern null pointer constant.

### Q5: void\*?

A: Generic pointer, cannot dereference.

### Q6: Pointer arithmetic?

A: Adding/subtracting to move through memory.

### Q7: const int \*p?

A: Pointer to const int.

### Q8: int \*const p?

A: Const pointer to int.

### Q9: Reference null?

A: No, must refer to valid object.

### Q10: Dangling pointer?

A: Points to freed memory.

## Topic 8: Dynamic Memory

---

Dynamic allocation creates objects at runtime. new allocates and calls constructor; delete frees. malloc/free don't call constructors. RAII ties resources to scope.

### Learn From

Memory references.

- C++ new/delete — <https://cplusplus.com/doc/tutorial/dynamic/>

### Key Concepts

- new and delete
- new[] and delete[]
- malloc, calloc, realloc, free
- Memory leaks
- Dangling pointers
- RAII principle
- Stack vs heap

### Problem 1

Allocate int=42 print free.

```
#include <iostream>
int main(){int *p=new int(42);std::cout<<*p<<std::endl;delete p;return 0;}
```

*Expected output:*

42

*Learning points:*

### Problem 2

Allocate array 5 ints.

```
#include <iostream>
int main(){int *arr=new int[5]{1,2,3,4,5};for(int i=0;i<5;i++)std::cout<<arr[i]<<std::endl;delete[] arr;return 0;}
```

*Expected output:*

1  
2  
3  
4  
5

*Learning points:*

### Problem 3

Malloc allocate int=99.

```
#include <iostream>
#include <cstdlib>
int main(){int *p=(int*)malloc(sizeof(int));*p=99;std::cout<<*p<<std::endl;free(p);return 0;}
```

*Expected output:*

99

*Learning points:*

## Problem 4

Allocate double=2.718.

```
#include <iostream>
int main(){double *p=new double(2.718);std::cout<<*p<<std::endl;delete p;return 0;}
```

*Expected output:*

2.718

*Learning points:*

## Problem 5

Malloc array realloc.

```
#include <iostream>
#include <cstdlib>
int main(){int *arr=(int*)malloc(3*sizeof(int));arr[0]=1;arr[1]=2;arr[2]=3;arr=(int*)realloc(arr,5*sizeof(int))}
```

*Expected output:*

1  
2  
3  
4  
5

*Learning points:*

## Problem 6

Delete then nullptr.

```
#include <iostream>
int main(){int *p=new int(10);delete p;p=nullptr;if(p==nullptr)std::cout<<"null"<<std::endl;return 0;}
```

*Expected output:*

null

*Learning points:*

## Problem 7

Allocate char array Hi.

```
#include <iostream>
int main(){char *str=new char[3];str[0]='H';str[1]='i';str[2]='\0';std::cout<<str<<std::endl;delete[] str;return 0;}
```

*Expected output:*

Hi

*Learning points:*

## Problem 8

Sum 3 doubles.

```
#include <iostream>
int main(){double *arr=new double[3]{1.1,2.2,3.3};double sum=0;for(int i=0;i<3;i++)sum+=arr[i];std::cout<<sum<<endl;return 0;}
```

*Expected output:*

6.6

*Learning points:*

## Problem 9

Memory leak demo.

```
#include <iostream>
int main(){int *p=new int(5);std::cout<<*p<<std::endl;return 0;}
```

*Expected output:*

5

*Learning points:*

## Problem 10

Malloc char sizeof.

```
#include <iostream>
#include <cstdlib>
int main(){char *p=(char*)malloc(sizeof(char));std::cout<<sizeof(*p)<<std::endl;free(p);return 0;}
```

*Expected output:*

1

*Learning points:*

## Quick Questions & Answers

### Q1: new vs malloc?

A: new calls constructor; malloc does not.

### Q2: delete[] for arrays?

A: Calls destructors for all elements.

### Q3: Memory leak?

A: Allocated memory never freed.

### Q4: RAI?

A: Resource lifetime tied to scope.

### Q5: Double free?

A: Undefined behavior.

### Q6: Dangling pointer?

A: Points to freed memory.

### Q7: Stack vs heap?

A: Stack automatic/fast; heap manual/larger.

**Q8: calloc vs malloc?**

A: Initializes to zero.

**Q9: free on new?**

A: Undefined behavior.

**Q10: Smart pointer?**

A: Object managing memory automatically.



## Topic 9: Strings (std::string)

---

std::string manages memory with rich methods. Concatenate with +, find with find(), substr() extracts, compare with ==.

### Learn From

String references.

- C++ std::string — <https://cplusplus.com/reference/string/string/>

### Key Concepts

- Constructors
- length() and size()
- Concatenation +
- substr(pos, len)
- find() and rfind()
- Comparison operators
- at() and [] access
- c\_str() conversion

### Problem 1

Create Hello and print.

```
#include <iostream>
#include <string>
int main(){std::string s="Hello";std::cout<<s<<std::endl;return 0;}
```

*Expected output:*

Hello

*Learning points:*

### Problem 2

Print length of Programming.

```
#include <iostream>
#include <string>
int main(){std::string s="Programming";std::cout<<s.length()<<std::endl;return 0;}
```

*Expected output:*

11

*Learning points:*

### Problem 3

Concatenate strings.

```
#include <iostream>
#include <string>
int main(){std::string s="Hello"+std::string(" World");std::cout<<s<<std::endl;return 0;}
```

*Expected output:*

Hello World

*Learning points:*

## Problem 4

Substring from Hello World.

```
#include <iostream>
#include <string>
int main(){std::string s="Hello World";std::cout<<s.substr(6,5)<<std::endl;return 0;}
```

*Expected output:*

World

*Learning points:*

## Problem 5

Find W in Hello World.

```
#include <iostream>
#include <string>
int main(){std::string s="Hello World";std::cout<<s.find('W')<<std::endl;return 0;}
```

*Expected output:*

6

*Learning points:*

## Problem 6

Compare strings.

```
#include <iostream>
#include <string>
int main(){std::string a="abc",b="abd";std::cout<<(a==b?"equal":"not equal")<<std::endl;return 0;}
```

*Expected output:*

not equal

*Learning points:*

## Problem 7

Access third char of Hello.

```
#include <iostream>
#include <string>
int main(){std::string s="Hello";std::cout<<s[2]<<std::endl;return 0;}
```

*Expected output:*

l

*Learning points:*

## Problem 8

Append ! to Hello.

```
#include <iostream>
#include <string>
int main(){std::string s="Hello";s+="!";std::cout<<s<<std::endl;return 0;}
```

*Expected output:*

Hello!

*Learning points:*

## Problem 9

Convert to C-string.

```
#include <iostream>
#include <string>
#include <cstring>
int main(){std::string s="Hi";const char *cs=s.c_str();for(int i=0;i<(int)strlen(cs);i++)std::cout<<cs[i]<<std::endl;return 0;}
```

*Expected output:*

H

i

*Learning points:*

## Problem 10

Check starts with H.

```
#include <iostream>
#include <string>
int main(){std::string s="Hello";std::cout<<(s.find('H')==0?"yes":"no")<<std::endl;return 0;}
```

*Expected output:*

yes

*Learning points:*

## Quick Questions & Answers

**Q1: size() vs length()?**

A: Identical.

**Q2: substr() bad position?**

A: std::out\_of\_range.

**Q3: find() not found?**

A: std::string::npos.

**Q4: Concat C-string + string?**

A: Yes, using +.

**Q5: at() vs []?**

A: at() bounds checks; [] does not.

**Q6: c\_str()?**

A: Returns const pointer to C-string.

**Q7: Clear string?**

A: clear() or assign empty.

**Q8: String holds null chars?**

A: Yes but c\_str() expects null-terminated.

**Q9: == vs compare()?**

A: == returns bool; compare() returns 0/neg/pos.

**Q10: Max string size?**

A: Limited by size\_t.

## Topic 10: Structs & Enums

---

Structs group data. Enums define named constants. Unions share memory. typedef aliases types. Bitfields pack flags into bits.

### Learn From

Struct references.

- C++ Structs — <https://cplusplus.com/doc/tutorial/structures/>

### Key Concepts

- Struct declaration
- Struct access
- Enum declaration
- Scoped enums
- Union memory
- typedef and using
- Bitfields

### Problem 1

Struct Point x,y.

```
#include <iostream>
struct Point{int x;int y;};
int main(){Point p={3,7};std::cout<<p.x<<" "<<p.y<<std::endl;return 0;}
```

*Expected output:*

3 7

*Learning points:*

### Problem 2

Enum Color GREEN=1.

```
#include <iostream>
enum Color{RED, GREEN, BLUE};
int main(){std::cout<<GREEN<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

### Problem 3

Union int/float.

```
#include <iostream>
union Data{int i;float f;};
int main(){Data d;d.i=42;std::cout<<d.i<<std::endl;d.f=3.14f;std::cout<<d.f<<std::endl;return 0;}
```

*Expected output:*

42  
3.14

*Learning points:*

## Problem 4

typedef alias.

```
#include <iostream>
typedef int Integer;
int main(){Integer x=10;std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

## Problem 5

Bitfield flags.

```
#include <iostream>
struct Flags{unsigned int active:1;unsigned int mode:2;};
int main(){Flags f={1,3};std::cout<<f.active<<" "<<f.mode<<std::endl;return 0;}
```

*Expected output:*

1 3

*Learning points:*

## Problem 6

Scoped enum class.

```
#include <iostream>
enum class Direction{North,South,East,West};
int main(){Direction d=Direction::North;std::cout<<static_cast<int>(d)<<std::endl;return 0;}
```

*Expected output:*

0

*Learning points:*

## Problem 7

Struct Rectangle area.

```
#include <iostream>
struct Rectangle{int w;int h;};
int main(){Rectangle r={5,3};std::cout<<r.w*r.h<<std::endl;return 0;}
```

*Expected output:*

15

*Learning points:*

## Problem 8

Struct Student print.

```
#include <iostream>
#include <string>
struct Student{std::string name;int grade;};
int main(){Student s={"Alice",95};std::cout<<s.name<<" "<<s.grade<<std::endl;return 0;}
```

*Expected output:*

Alice 95

*Learning points:*

## Problem 9

Union as bytes.

```
#include <iostream>
union IntBytes{int i;unsigned char bytes[4];};
int main(){IntBytes u;u.i=0x41424344;for(int i=0;i<4;i++)std::cout<<(int)u.bytes[i]<<std::endl;return 0;}
```

*Expected output:*

68

67

66

65

*Learning points:*

## Problem 10

using alias double.

```
#include <iostream>
using F64=double;
int main(){F64 pi=3.14159;std::cout<<pi<<std::endl;return 0;}
```

*Expected output:*

3.14159

*Learning points:*

## Quick Questions & Answers

### Q1: struct vs class?

A: Struct public default; class private default.

### Q2: enum purpose?

A: Named integer constants.

### Q3: scoped enum?

A: Values scoped within enum.

### Q4: union memory?

A: All members share same memory.

### Q5: bitfield?

A: Struct member with specified bits.

**Q6: typedef purpose?**

A: Alias for existing type.

**Q7: Struct own type?**

A: Only as pointer.

**Q8: Empty struct size?**

A: At least 1 byte.

**Q9: Pass struct by value?**

A: Yes, entire struct copied.

**Q10: Default enum values?**

A: From 0, incrementing.



# Topic 11: OOP: Classes & Objects

---

OOP builds around classes bundling data and behavior. Constructors initialize; destructors clean up. `this` pointer refers to current object.

## Learn From

Classes references.

- C++ Classes — <https://cplusplus.com/doc/tutorial/classes/>

## Key Concepts

- Class with `public/private/protected`
- Constructors
- Destructor
- `this` pointer
- Member functions
- Access specifiers
- Struct vs class

### Problem 1

Class Animal speak.

```
#include <iostream>
#include <string>
class Animal{public:std::string name;void speak(){std::cout<<name<<std::endl;}};
int main(){Animal a;a.name="Dog";a.speak();return 0;}
```

*Expected output:*

Dog

*Learning points:*

### Problem 2

Class Box default constructor.

```
#include <iostream>
class Box{public:int volume;Box():volume(0){}};
int main(){Box b;std::cout<<b.volume<<std::endl;return 0;}
```

*Expected output:*

0

*Learning points:*

### Problem 3

Circle with area.

```
#include <iostream>
#include <cmath>
class Circle{public:double radius;Circle(double r):radius(r){}double area(){return 3.14159*radius*radius;}};
int main(){Circle c(5);std::cout<<c.area()<<std::endl;return 0;}
```

*Expected output:*

78.5397

*Learning points:*

## Problem 4

Counter increment/get.

```
#include <iostream>
class Counter{private:int count;public:Counter():count(0){}void increment(){count++;}int get(){return count;}};
int main(){Counter c;c.increment();c.increment();std::cout<<c.get()<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

## Problem 5

This pointer setter.

```
#include <iostream>
class MyClass{public:int value;MyClass &setValue(int v){this->value=v;return *this;}};
int main(){MyClass obj;obj.setValue(42);std::cout<<obj.value<<std::endl;return 0;}
```

*Expected output:*

42

*Learning points:*

## Problem 6

Copy constructor.

```
#include <iostream>
class Vec2{public:int x,y;Vec2(int x,int y):x(x),y(y){}Vec2(const Vec2 &o):x(o.x),y(o.y){}};
int main(){Vec2 a(1,2);Vec2 b=a;std::cout<<b.x<<" "<<b.y<<std::endl;return 0;}
```

*Expected output:*

1 2

*Learning points:*

## Problem 7

Static member count.

```
#include <iostream>
class Obj{public:static int count;Obj(){count++;}};
int Obj::count=0;
int main(){Obj a,b,c;std::cout<<Obj::count<<std::endl;return 0;}
```

*Expected output:*

3

*Learning points:*

## Problem 8

Rectangle area/perimeter.

```
#include <iostream>
class Rectangle{public:int w,h;Rectangle(int w,int h):w(w),h(h){}int area(){return w*h;}int perimeter(){return 2*w+2*h;}
int main(){Rectangle r(4,5);std::cout<<r.area()<<std::endl;std::cout<<r.perimeter()<<std::endl;return 0;}
```

*Expected output:*

20  
18

*Learning points:*

## Problem 9

Const member function.

```
#include <iostream>
class Point{public:int x,y;Point(int x,int y):x(x),y(y){}void print()const{std::cout<<x<<" "<<y<<std::endl;}}
int main(){const Point p(3,4);p.print();return 0;}
```

*Expected output:*

3, 4

*Learning points:*

## Problem 10

Singleton pattern.

```
#include <iostream>
class Singleton{public:static Singleton& getInstance(){static Singleton instance;return instance;}void show()const{std::cout<<"Singleton\n";}
int main(){Singleton::getInstance().show();return 0;}
```

*Expected output:*

Singleton

*Learning points:*

## Quick Questions & Answers

### Q1: Public vs private?

A: Public anywhere; private within class.

### Q2: this pointer?

A: Hidden pointer to current object.

### Q3: Constructor?

A: Called on object creation.

### Q4: Copy constructor?

A: Creates new object as copy.

### Q5: Destructor?

A: Called when object goes out of scope.

### Q6: Private constructor?

A: Yes, for singleton/factory.

**Q7: const member function?**

A: Promises not to modify members.

**Q8: struct vs class?**

A: Struct public default; class private.

**Q9: static member?**

A: Shared by all instances.

**Q10: No constructor?**

A: Compiler provides default.

## Topic 12: OOP: Inheritance & Polymorphism

---

Inheritance enables code reuse. Polymorphism allows base pointers to call derived methods via virtual functions. override ensures correct overriding. Pure virtual makes abstract classes.

### Learn From

Inheritance references.

- C++ Inheritance — <https://cplusplus.com/doc/tutorial/inheritance/>

### Key Concepts

- Base and derived
- Single inheritance
- Virtual functions
- override keyword
- Pure virtual and abstract
- Constructor chaining
- Slicing problem
- Multiple inheritance

### Problem 1

Animal virtual speak Dog Woof.

```
#include <iostream>
class Animal{public:virtual void speak(){std::cout<<"..."<<std::endl;}};
class Dog:public Animal{public:void speak()override{std::cout<<"Woof"<<std::endl;}};
int main(){Dog d;d.speak();return 0;}
```

*Expected output:*

Woof

*Learning points:*

### Problem 2

Base pointer derived method.

```
#include <iostream>
class Base{public:virtual void show(){std::cout<<"Base"<<std::endl;}};
class Derived:public Base{public:void show()override{std::cout<<"Derived"<<std::endl;}};
int main(){Base *p=new Derived();p->show();delete p;return 0;}
```

*Expected output:*

Derived

*Learning points:*

### Problem 3

Abstract Shape pure virtual area.

```
#include <iostream>
class Shape{public:virtual double area()=0;};
class Circle:public Shape{public:double r;Circle(double r):r(r){}double area()override{return 3.14159*r*r;}};
int main(){Circle c(5);std::cout<<c.area()<<std::endl;return 0;}
```

*Expected output:*

78.5397

*Learning points:*

## Problem 4

Shape->Rectangle area.

```
#include <iostream>
class Shape{public:virtual double area()=0;virtual ~Shape(){};};
class Rectangle:public Shape{public:double w,h;Rectangle(double w,double h):w(w),h(h){}double area()override{
int main(){Rectangle r(3,4);std::cout<<r.area()<<std::endl;return 0;}
```

*Expected output:*

12

*Learning points:*

## Problem 5

Constructor chaining.

```
#include <iostream>
class Base{public:Base(){std::cout<<"Base"<<std::endl;}};
class Derived:public Base{public:Derived(){std::cout<<"Derived"<<std::endl;}};
int main(){Derived d;return 0;}
```

*Expected output:*

Base

Derived

*Learning points:*

## Problem 6

Slicing problem.

```
#include <iostream>
class Base{public:virtual void type(){std::cout<<"Base"<<std::endl;}};
class Derived:public Base{public:void type()override{std::cout<<"Derived"<<std::endl;}};
int main(){Derived d;Base b=d;b.type();return 0;}
```

*Expected output:*

Base

*Learning points:*

## Problem 7

Virtual destructor.

```
#include <iostream>
class Base{public:virtual ~Base(){std::cout<<"~Base"<<std::endl;}};
class Derived:public Base{public:~Derived(){std::cout<<"~Derived"<<std::endl;}};
int main(){Base *p=new Derived();delete p;return 0;}
```

*Expected output:*

~Derived  
~Base

*Learning points:*

## Problem 8

Vehicle->Car printInfo.

```
#include <iostream>
#include <string>
class Vehicle{public:std::string brand;Vehicle(std::string b):brand(b){}virtual void printInfo(){std::cout<<b<<std::endl;}};
class Car:public Vehicle{public:int doors;Car(std::string b,int d):Vehicle(b),doors(d){}void printInfo()override{std::cout<<b<<std::endl;<<doors<<std::endl;}};
int main(){Car c("Toyota",4);c.printInfo();return 0;}
```

*Expected output:*

Toyota 4

*Learning points:*

## Problem 9

Dynamic\_cast downcast.

```
#include <iostream>
class Base{public:virtual ~Base(){};};
class Derived:public Base{public:int x=10;};
int main(){Base *b=new Derived();Derived *d=dynamic_cast<Derived*>(b);if(d)std::cout<<d->x<<std::endl;delete b;return 0;}
```

*Expected output:*

10

*Learning points:*

## Problem 10

Pure virtual interface.

```
#include <iostream>
class Interface{public:virtual void execute()=0;};
class Implementation:public Interface{public:void execute()override{std::cout<<"Done"<<std::endl;}};
int main(){Implementation i;i.execute();return 0;}
```

*Expected output:*

Done

*Learning points:*

## Quick Questions & Answers

### Q1: Virtual functions?

A: Enable runtime polymorphism.

### Q2: override?

A: Ensures correct overriding.

### Q3: Pure virtual?

A: Virtual with =0, no base impl.

### Q4: Abstract class?

A: Has pure virtual, cannot instantiate.

### Q5: Virtual destructor?

A: Ensures derived dtor called via base ptr.

### Q6: Object slicing?

A: Derived loses members when assigned to base.

### Q7: Multiple inheritance?

A: Yes, from multiple bases.

### Q8: Diamond problem?

A: Ambiguity from shared base.

### Q9: dynamic\_cast?

A: Safely downcasts at runtime.

### Q10: Virtual vs non-virtual?

A: Virtual dynamic dispatch; non-virtual static.



## Topic 13: Operator Overloading

Operator overloading redefines operators for user-defined types. Overload arithmetic, stream operators, subscript, and function call operators.

### Learn From

Operator references.

- C++ Operators — <https://cplusplus.com/doc/tutorial/operators/>

### Key Concepts

- Overload + - \* /
- Overload << >>
- Overload []
- Overload ()
- Friend functions
- Unary overloading
- Comparison overloading

### Problem 1

Vec2 overloaded +.

```
#include <iostream>
class Vec2{public:int x,y;Vec2(int x,int y):x(x),y(y){}Vec2 operator+(const Vec2 &o)const{return Vec2(x+o.x,y+o.y);}
int main(){Vec2 a(1,2),b(3,4);Vec2 c=a+b;std::cout<<c.x<<" "<<c.y<<std::endl;return 0;}
```

*Expected output:*

4 6

*Learning points:*

### Problem 2

Overloaded << for Point.

```
#include <iostream>
class Point{public:int x,y;Point(int x,int y):x(x),y(y){}friend std::ostream &operator<<(std::ostream &os,const Point &p){os<<p.x<<","<<p.y<<std::endl;return os;}
int main(){Point p(3,4);std::cout<<p<<std::endl;return 0;}
```

*Expected output:*

(3,4)

*Learning points:*

### Problem 3

IntArray overloaded [].

```
#include <iostream>
#include <vector>
class IntArray{std::vector<int> data;public:IntArray(std::initializer_list<int> l):data(l){}int &operator[](int i){return data[i];}
int main(){IntArray arr={10,20,30};std::cout<<arr[1]<<std::endl;return 0;}
```

*Expected output:*

20

*Learning points:*

## Problem 4

Adder functor ().

```
#include <iostream>
class Adder{public:int operator()(int a,int b){return a+b;}};
int main(){Adder add;std::cout<<add(3,4)<<std::endl;return 0;}
```

*Expected output:*

7

*Learning points:*

## Problem 5

Counter prefix ++.

```
#include <iostream>
class Counter{public:int val;Counter(int v):val(v){}Counter &operator++(){val++;return *this;}};
int main(){Counter c(5);++c;std::cout<<c.val<<std::endl;return 0;}
```

*Expected output:*

6

*Learning points:*

## Problem 6

Overloaded ==.

```
#include <iostream>
class Vec2{public:int x,y;Vec2(int x,int y):x(x),y(y){}bool operator==(const Vec2 &o)const{return x==o.x&&y==o.y;}};
int main(){Vec2 a(1,2),b(1,2);std::cout<<(a==b)<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

## Problem 7

Scalar multiplication \*.

```
#include <iostream>
class Vec2{public:int x,y;Vec2(int x,int y):x(x),y(y){}Vec2 operator*(int s)const{return Vec2(x*s,y*s);}};
int main(){Vec2 a(2,3);Vec2 b=a*3;std::cout<<b.x<<" "<<b.y<<std::endl;return 0;}
```

*Expected output:*

6 9

*Learning points:*

## Problem 8

Unary minus.

```
#include <iostream>
class Vec2{public:int x,y;Vec2(int x,int y):x(x),y(y){}Vec2 operator-()const{return Vec2(-x,-y);}};
int main(){Vec2 a(3,4);Vec2 b=-a;std::cout<<b.x<<" "<<b.y<<std::endl;return 0;}
```

*Expected output:*

-3 -4

*Learning points:*

## Problem 9

Overloaded !=.

```
#include <iostream>
class Vec2{public:int x,y;Vec2(int x,int y):x(x),y(y){}bool operator==(const Vec2 &o)const{return x==o.x&&y==o.y};}
int main(){Vec2 a(1,2),b(3,4);std::cout<<(a!=b)<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

## Problem 10

Overloaded < for sorting.

```
#include <iostream>
#include <algorithm>
#include <vector>
class Score{public:int val;Score(int v):val(v){}bool operator<(const Score &o)const{return val<o.val;}};
int main(){std::vector<Score> v={Score(5),Score(1),Score(3)};std::sort(v.begin(),v.end());std::cout<<v[0].val<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

## Quick Questions & Answers

### Q1: Overload all?

A: No, :: . \* ? : cannot.

### Q2: Friend function?

A: Non-member with private access.

### Q3: Prefix vs postfix?

A: Prefix reference; postfix dummy int.

### Q4: Overload <<?

A: Yes, as friend.

### Q5: Functor?

A: Class with operator().

### Q6: [] return reference?

A: Yes, for read/write.

**Q7: Overload ==?**

A: Natural comparison.

**Q8: Built-in types?**

A: No, only user-defined.

**Q9: == return type?**

A: bool.

**Q10: << be friend?**

A: Yes, left is ostream.

# Topic 14: Templates

---

Templates enable generic programming. Function/class templates generate code per type. Specialization provides custom behavior.

## Learn From

Templates references.

- C++ Templates — <https://cplusplus.com/doc/tutorial/templates/>

## Key Concepts

- Function templates
- Class templates
- Template specialization
- Non-type parameters
- Template instantiation
- SFINAE basics
- Variadic templates

### Problem 1

Template max.

```
#include <iostream>
template<typename T>
T maxValue(T a,T b){return (a>b)?a:b;}
int main(){std::cout<<maxValue(3,7)<<std::endl;std::cout<<maxValue(2.5,1.5)<<std::endl;return 0;}
```

*Expected output:*

7  
2.5

*Learning points:*

### Problem 2

Template swap.

```
#include <iostream>
template<typename T>
void mySwap(T &a,T &b){T temp=a;a=b;b=temp;}
int main(){int x=1,y=2;mySwap(x,y);std::cout<<x<<" "<<y<<std::endl;return 0;}
```

*Expected output:*

2 1

*Learning points:*

### Problem 3

Class template Pair.

```
#include <iostream>
template<typename T,typename U>
class Pair{public:T first;U second;Pair(T f,U s):first(f),second(s){}};
int main(){Pair<int,double> p(42,3.14);std::cout<<p.first<<" "<<p.second<<std::endl;return 0;}
```

*Expected output:*

42 3.14

*Learning points:*

## Problem 4

Template print array.

```
#include <iostream>
template<typename T>
void printArray(T arr[],int size){for(int i=0;i<size;i++)std::cout<<arr[i]<<std::endl;}
int main(){int arr[]={1,2,3};printArray(arr,3);return 0;}
```

*Expected output:*

1  
2  
3

*Learning points:*

## Problem 5

Template specialization const char\*.

```
#include <iostream>
#include <cstring>
template<typename T>
T minValue(T a,T b){return (a<b)?a:b;}
template<>
const char* minValue<const char*>(const char* a,const char* b){return (strcmp(a,b)<0)?a:b;}
int main(){std::cout<<minValue("apple","banana")<<std::endl;return 0;}
```

*Expected output:*

apple

*Learning points:*

## Problem 6

Class template Stack.

```
#include <iostream>
#include <vector>
template<typename T>
class Stack{std::vector<T> data;public:void push(T v){data.push_back(v);}void pop(){data.pop_back();}T top(){
int main(){Stack<int> s;s.push(10);s.push(20);std::cout<<s.top()<<std::endl;s.pop();std::cout<<s.top()<<std::endl;return 0;}
```

*Expected output:*

20  
10

*Learning points:*

## Problem 7

Template abs.

```
#include <iostream>
template<typename T>
T myAbs(T x){return (x<0)?-x:x;}
int main(){std::cout<<myAbs(-5)<<std::endl;std::cout<<myAbs(-3.14)<<std::endl;return 0;}
```

*Expected output:*

5  
3.14

*Learning points:*

## Problem 8

Template first element.

```
#include <iostream>
template<typename T>
T first(T arr[]){return arr[0];}
int main(){int arr[]={99,2,3};std::cout<<first(arr)<<std::endl;return 0;}
```

*Expected output:*

99

*Learning points:*

## Problem 9

Template bool specialization.

```
#include <iostream>
template<typename T>
void printVal(T v){std::cout<<v<<std::endl;}
template<>
void printVal<bool>(bool v){std::cout<<(v?"true":"false")<<std::endl;}
int main(){printVal(42);printVal(true);printVal(false);return 0;}
```

*Expected output:*

42  
true  
false

*Learning points:*

## Problem 10

Non-type template power of 2.

```
#include <iostream>
template<int N>
int powerOf2(){return 1<<N;}
int main(){std::cout<<powerOf2<3>()<<std::endl;std::cout<<powerOf2<0>()<<std::endl;return 0;}
```

*Expected output:*

8

1

*Learning points:*

## Quick Questions & Answers

### **Q1: Template?**

A: Blueprint for generic code.

### **Q2: Specialization?**

A: Custom impl for specific type.

### **Q3: When instantiated?**

A: At compile time.

### **Q4: Non-type parameter?**

A: Takes a value, not type.

### **Q5: SFINAE?**

A: Substitution Failure Not Error.

### **Q6: Templates in headers?**

A: Yes, needed at instantiation.

### **Q7: typename keyword?**

A: Indicates dependent type name.

### **Q8: Variadic template?**

A: Variable number of arguments.

### **Q9: Different members?**

A: Yes, specialization can differ.

### **Q10: typename vs class?**

A: Identical for type params.



## Topic 15: STL: Containers

---

STL provides ready-made containers. vector is dynamic array; map stores key-value pairs; set stores unique elements; stack/queue are adapters.

### Learn From

Container references.

- C++ Containers — <https://cplusplus.com/reference/stl/>

### Key Concepts

- `std::vector`
- `std::list`
- `std::map`
- `std::set`
- `std::stack`
- `std::queue`
- `std::pair`
- Container operations

### Problem 1

Vector push 4 print size.

```
#include <iostream>
#include <vector>
int main(){std::vector<int> v={1,2,3};v.push_back(4);std::cout<<v.size()<<std::endl;return 0;}
```

*Expected output:*

4

*Learning points:*

### Problem 2

Map insert key-value.

```
#include <iostream>
#include <map>
#include <string>
int main(){std::map<std::string,int> m;m["one"]=1;m["two"]=2;std::cout<<m["one"]<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

### Problem 3

Set insert unique.

```
#include <iostream>
#include <set>
int main(){std::set<int> s;s.insert(3);s.insert(1);s.insert(3);std::cout<<s.size()<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

### Problem 4

Stack push/pop.

```
#include <iostream>
#include <stack>
int main(){std::stack<int> s;s.push(1);s.push(2);s.push(3);s.pop();std::cout<<s.top()<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

### Problem 5

Queue push/pop.

```
#include <iostream>
#include <queue>
int main(){std::queue<int> q;q.push(10);q.push(20);q.push(30);q.pop();std::cout<<q.front()<<std::endl;return 0;}
```

*Expected output:*

20

*Learning points:*

### Problem 6

Pair creation.

```
#include <iostream>
#include <utility>
#include <string>
int main(){std::pair<int,std::string> p(42,"hello");std::cout<<p.first<<" "<<p.second<<std::endl;return 0;}
```

*Expected output:*

42 hello

*Learning points:*

### Problem 7

Vector [] access.

```
#include <iostream>
#include <vector>
int main(){std::vector<int> v={10,20,30};std::cout<<v[0]<<" "<<v[2]<<std::endl;return 0;}
```

*Expected output:*

10 30

*Learning points:*

## Problem 8

Vector at().

```
#include <iostream>
#include <vector>
int main(){std::vector<int> v={1,2,3};std::cout<<v.at(1)<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

## Problem 9

Map find check.

```
#include <iostream>
#include <map>
#include <string>
int main(){std::map<int,std::string> m;m[1]="one";if(m.find(1)!=m.end())std::cout<<m[1]<<std::endl;return 0;}
```

*Expected output:*

one

*Learning points:*

## Problem 10

Vector clear empty.

```
#include <iostream>
#include <vector>
int main(){std::vector<int> v={1,2,3};v.clear();std::cout<<(v.empty()?"empty":"not empty")<<std::endl;return 0;}
```

*Expected output:*

empty

*Learning points:*

## Quick Questions & Answers

### Q1: vector vs array?

A: Dynamic vs fixed-size.

### Q2: Vector access?

A:  $O(1)$  random access.

### Q3: Map insert?

A:  $O(\log n)$ .

### Q4: find not found?

A: Returns end().

### Q5: Set duplicates?

A: No, use multiset.

### Q6: stack vs queue?

A: LIFO vs FIFO.

**Q7: push\_back?**

A: Amortized  $O(1)$ .

**Q8: pair?**

A: Holds two values.

**Q9: multimap?**

A: Multiple values per key.

**Q10: begin vs end?**

A: First element vs past last.

## Topic 16: STL: Iterators & Algorithms

Iterators point to elements with uniform interface. Algorithms work on iterator ranges. Lambdas are common arguments.

### Learn From

Iterator references.

- C++ Algorithms — <https://en.cppreference.com/w/cpp/algorithm>

### Key Concepts

- Iterator types
- begin() and end()
- sort, find, transform
- for\_each, count\_if
- reverse, copy
- Lambdas with algorithms
- Iterator invalidation

### Problem 1

Sort vector.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={5,3,1,4,2};std::sort(v.begin(),v.end());for(int x:v)std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

```
1
2
3
4
5
```

*Learning points:*

### Problem 2

Find 7.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={3,7,1,9};auto it=std::find(v.begin(),v.end(),7);std::cout<<(it!=v.end())<<std::endl;return 0;}
```

*Expected output:*

```
1
```

*Learning points:*

### Problem 3

Transform double.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={1,2,3};std::vector<int> r(3);std::transform(v.begin(),v.end(),r.begin(),[](int
```

*Expected output:*

2  
4  
6

*Learning points:*

### Problem 4

Count\_if >5.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={3,8,1,9,4};int c=std::count_if(v.begin(),v.end(),[](int x){return x>5;});std::cout<<c<<std::endl;
```

*Expected output:*

2

*Learning points:*

### Problem 5

Reverse vector.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={1,2,3,4,5};std::reverse(v.begin(),v.end());for(int x:v)std::cout<<x<<std::endl;
```

*Expected output:*

5  
4  
3  
2  
1

*Learning points:*

### Problem 6

For\_each print.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={10,20,30};std::for_each(v.begin(),v.end(),[](int x){std::cout<<x<<std::endl;});}
```

*Expected output:*

10  
20  
30

*Learning points:*

## Problem 7

Accumulate sum.

```
#include <iostream>
#include <vector>
#include <numeric>
int main(){std::vector<int> v={1,2,3,4,5};int s=std::accumulate(v.begin(),v.end(),0);std::cout<<s<<std::endl;
```

*Expected output:*

15

*Learning points:*

## Problem 8

Copy vector.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> src={1,2,3};std::vector<int> dst(3);std::copy(src.begin(),src.end(),dst.begin());
```

*Expected output:*

1  
2  
3

*Learning points:*

## Problem 9

Max element.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={3,1,4,1,5};auto it=std::max_element(v.begin(),v.end());std::cout<<*it<<std::endl;
```

*Expected output:*

5

*Learning points:*

## Problem 10

Sort descending lambda.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={3,1,4,1,5};std::sort(v.begin(),v.end(),[](int a,int b){return a>b;});for(int x
```

*Expected output:*

5  
4  
3  
1  
1

*Learning points:*

## Quick Questions & Answers

### **Q1: Iterator?**

A: Points to container element.

### **Q2: begin()?**

A: First element.

### **Q3: end()?**

A: Past last element.

### **Q4: sort complexity?**

A:  $O(n \log n)$ .

### **Q5: find not found?**

A: Returns end().

### **Q6: transform?**

A: Apply function to elements.

### **Q7: Lambdas with STL?**

A: Yes, as comparators/predicates.

### **Q8: accumulate?**

A: Sum or binary operation.

### **Q9: Invalidation?**

A: Iterators point to invalid memory.

### **Q10: Categories?**

A: Input, output, forward, bidirectional, random-access.



## Topic 17: Lambda & Modern C++

---

C++11 introduced lambdas, auto, range-for, nullptr, and initializer lists for cleaner, safer code.

### Learn From

Modern C++ references.

- cppreference Lambda — <https://en.cppreference.com/w/cpp/language/lambda>

### Key Concepts

- Lambda [capture](params){body}
- Capture by value/reference
- auto deduction
- Range-based for
- nullptr
- Initializer lists
- std::function

### Problem 1

Lambda prints Hello Lambda.

```
#include <iostream>
int main(){auto greet=[](){std::cout<<"Hello Lambda"<<std::endl;};greet();return 0;}
```

*Expected output:*

Hello Lambda

*Learning points:*

### Problem 2

Lambda adds two ints.

```
#include <iostream>
int main(){auto add=[](int a,int b){return a+b;};std::cout<<add(3,4)<<std::endl;return 0;}
```

*Expected output:*

7

*Learning points:*

### Problem 3

Auto deduction.

```
#include <iostream>
int main(){auto x=42;auto y=3.14;std::cout<<x<<std::endl;std::cout<<y<<std::endl;return 0;}
```

*Expected output:*

42

3.14

*Learning points:*

## Problem 4

Range-based for.

```
#include <iostream>
#include <vector>
int main(){std::vector<int> v={1,2,3,4,5};for(int x:v)std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

1  
2  
3  
4  
5

*Learning points:*

## Problem 5

Lambda capture value.

```
#include <iostream>
int main(){int f=3;auto m=[f](int x){return x*f;};std::cout<<m(5)<<std::endl;return 0;}
```

*Expected output:*

15

*Learning points:*

## Problem 6

Lambda capture reference.

```
#include <iostream>
int main(){int c=0;auto inc=[&c]() {c++;};inc();inc();std::cout<<c<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

## Problem 7

nullptr check.

```
#include <iostream>
int main(){int *p=nullptr;if(p==nullptr)std::cout<<"null"<<std::endl;return 0;}
```

*Expected output:*

null

*Learning points:*

## Problem 8

Initializer list vector.

```
#include <iostream>
#include <vector>
int main(){std::vector<int> v={10,20,30};for(int x:v)std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

10  
20  
30

*Learning points:*

## Problem 9

Lambda comparator sort.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main(){std::vector<int> v={5,2,8,1};std::sort(v.begin(),v.end(),[](int a,int b){return a<b;});for(int x:v)
```

*Expected output:*

1  
2  
5  
8

*Learning points:*

## Problem 10

Lambda auto return.

```
#include <iostream>
int main(){auto sq=[](int x){return x*x;};std::cout<<sq(6)<<std::endl;return 0;}
```

*Expected output:*

36

*Learning points:*

## Quick Questions & Answers

### Q1: Lambda?

A: Anonymous function.

### Q2: Capture list?

A: Variables and how to capture.

### Q3: [x] vs [&x]?

A: Value vs reference.

### Q4: auto?

A: Deduces type.

### Q5: Range-for?

A: Iterates all elements.

**Q6: nullptr vs NULL?**

A: Type-safe vs 0.

**Q7: Initializer lists?**

A: Curly-brace initialization.

**Q8: [=] capture?**

A: All by value.

**Q9: Store lambda?**

A: auto or std::function.

**Q10: Return syntax?**

A: -> return\_type.

## Topic 18: Smart Pointers & RAII

---

Smart pointers automate memory. `unique_ptr` exclusive; `shared_ptr` shared; `weak_ptr` observes. RAII ties resources to scope.

### Learn From

Smart pointer references.

- C++ Smart Pointers — [https://en.cppreference.com/w/cpp/memory/unique\\_ptr](https://en.cppreference.com/w/cpp/memory/unique_ptr)

### Key Concepts

- `unique_ptr` exclusive
- `shared_ptr` shared
- `weak_ptr` observer
- `make_unique`/`make_shared`
- RAII cleanup
- Circular references
- Custom deleters

### Problem 1

`unique_ptr` manage int.

```
#include <iostream>
#include <memory>
int main(){std::unique_ptr<int> p=std::make_unique<int>(42);std::cout<<*p<<std::endl;return 0;}
```

*Expected output:*

42

*Learning points:*

### Problem 2

`shared_ptr` share string.

```
#include <iostream>
#include <memory>
#include <string>
int main(){auto p1=std::make_shared<std::string>("Hello");auto p2=p1;std::cout<<*p1<<std::endl;std::cout<<p1->get_value();}
```

*Expected output:*

Hello

2

*Learning points:*

### Problem 3

`weak_ptr` observe.

```
#include <iostream>
#include <memory>
int main(){std::weak_ptr<int> wp;{auto sp=std::make_shared<int>(10);wp=sp;std::cout<<wp.expired()<<std::endl;}}
```

*Expected output:*

0  
1

*Learning points:*

## Problem 4

unique\_ptr array.

```
#include <iostream>
#include <memory>
int main(){auto arr=std::make_unique<int[]>(3);arr[0]=10;arr[1]=20;arr[2]=30;for(int i=0;i<3;i++)std::cout<<a
```

*Expected output:*

10  
20  
30

*Learning points:*

## Problem 5

unique\_ptr move.

```
#include <iostream>
#include <memory>
int main(){auto p1=std::make_unique<int>(5);auto p2=std::move(p1);std::cout<<(p1==nullptr)<<std::endl;std::co
```

*Expected output:*

1  
5

*Learning points:*

## Problem 6

make\_shared pair.

```
#include <iostream>
#include <memory>
#include <utility>
int main(){auto p=std::make_shared<std::pair<int,int>>(1,2);std::cout<<p->first<<" "<<p->second<<std::endl;re
```

*Expected output:*

1 2

*Learning points:*

## Problem 7

RAII resource class.

```
#include <iostream>
#include <memory>
class R{public:R(){std::cout<<"acquired"<<std::endl;}~R(){std::cout<<"released"<<std::endl;}};
int main(){auto p=std::make_unique<R>();return 0;}
```

*Expected output:*

acquired  
released

*Learning points:*

## Problem 8

shared\_ptr custom deleter.

```
#include <iostream>
#include <memory>
int main(){auto d=[](int *p){std::cout<<"deleted"<<std::endl;delete p;};std::shared_ptr<int> p(new int(7),d);}
```

*Expected output:*

7  
deleted

*Learning points:*

## Problem 9

weak\_ptr lock.

```
#include <iostream>
#include <memory>
int main(){std::weak_ptr<int> wp;{auto sp=std::make_shared<int>(99);wp=sp;auto l=wp.lock();if(l)std::cout<<*l<<std::endl;}
```

*Expected output:*

99

*Learning points:*

## Problem 10

unique\_ptr nullptr check.

```
#include <iostream>
#include <memory>
int main(){std::unique_ptr<int> p;if(!p)std::cout<<"empty"<<std::endl;p=std::make_unique<int>(88);if(p)std::cout<<*p<<std::endl;}
```

*Expected output:*

empty  
88

*Learning points:*

## Quick Questions & Answers

### Q1: unique\_ptr vs shared\_ptr?

A: Exclusive vs shared ownership.

### Q2: weak\_ptr?

A: Observe without owning.

### Q3: make\_unique?

A: Creates unique\_ptr safely.

### Q4: make\_shared?

A: Creates shared\_ptr safely.

**Q5: Copy unique\_ptr?**

A: No, use std::move.

**Q6: Circular reference?**

A: shared\_ptrs prevent deallocation.

**Q7: RAI?**

A: Resources tied to scope.

**Q8: Last shared\_ptr?**

A: Object destroyed.

**Q9: weak\_ptr access?**

A: Call lock() first.

**Q10: Custom deleter?**

A: Defines cleanup function.



## Topic 19: Exception Handling

---

Exceptions handle runtime errors. throw raises; try wraps; catch handles. Propagates up call stack. noexcept means no throw.

### Learn From

Exception references.

- C++ Exceptions — <https://cplusplus.com/doc/tutorial/exceptions/>

### Key Concepts

- throw, try, catch
- catch-all (...)
- Standard exceptions
- Custom exceptions
- noexcept
- Stack unwinding
- Exception safety

### Problem 1

Throw int exception.

```
#include <iostream>
int main(){try{throw 42;}catch(int e){std::cout<<e<<std::endl;}return 0;}
```

*Expected output:*

42

*Learning points:*

### Problem 2

Throw string exception.

```
#include <iostream>
#include <string>
int main(){try{throw std::string("error");}catch(std::string &e){std::cout<<e<<std::endl;}return 0;}
```

*Expected output:*

error

*Learning points:*

### Problem 3

Throw runtime\_error.

```
#include <iostream>
#include <stdexcept>
int main(){try{throw std::runtime_error("boom");}catch(std::exception &e){std::cout<<e.what()<<std::endl;}return 0;}
```

*Expected output:*

boom

*Learning points:*

## Problem 4

Multiple catch blocks.

```
#include <iostream>
int main(){try{throw 3.14;}catch(int e){std::cout<<"int"<<std::endl;}catch(double e){std::cout<<"double"<<std::endl;}}
```

*Expected output:*

double

*Learning points:*

## Problem 5

Catch-all handler.

```
#include <iostream>
int main(){try{throw 'x';}catch(...){std::cout<<"caught"<<std::endl;}return 0;}
```

*Expected output:*

caught

*Learning points:*

## Problem 6

Throw negative.

```
#include <iostream>
#include <stdexcept>
void check(int n){if(n<0)throw std::invalid_argument("negative");}
int main(){try{check(-1);}catch(std::invalid_argument &e){std::cout<<e.what()<<std::endl;}return 0;}
```

*Expected output:*

negative

*Learning points:*

## Problem 7

Custom exception class.

```
#include <iostream>
#include <exception>
#include <string>
class MyErr:public std::exception{std::string m;public:MyErr(std::string s):m(s){}const char* what()const noexcept{return m.c_str();}
int main(){try{throw MyErr("custom");}catch(MyErr &e){std::cout<<e.what()<<std::endl;}return 0;}
```

*Expected output:*

custom

*Learning points:*

## Problem 8

Exception propagation.

```
#include <iostream>
#include <stdexcept>
void inner(){throw std::runtime_error("deep");}
void outer(){inner();}
int main(){try{outer();}catch(std::exception &e){std::cout<<e.what()<<std::endl;}return 0;}
```

*Expected output:*

deep

*Learning points:*

## Problem 9

noexcept specifier.

```
#include <iostream>
void safe()noexcept{std::cout<<"safe"<<std::endl;}
int main(){safe();return 0;}
```

*Expected output:*

safe

*Learning points:*

## Problem 10

Re-throw exception.

```
#include <iostream>
#include <stdexcept>
int main(){try{try{throw std::runtime_error("orig");}catch(...){throw;}}catch(std::exception &e){std::cout<<e.what()<<std::endl;}return 0;}
```

*Expected output:*

orig

*Learning points:*

## Quick Questions & Answers

**Q1: try-catch purpose?**

A: Handle exceptions.

**Q2: throw do?**

A: Raises exception.

**Q3: catch-all?**

A: catch(...) catches any.

**Q4: std::exception?**

A: Base class for exceptions.

**Q5: noexcept?**

A: Guarantees no throw.

**Q6: Stack unwinding?**

A: Destroying locals during propagation.

**Q7: Uncatched?**

A: `std::terminate()` called.

**Q8: Constructors throw?**

A: Yes.

**Q9: Exception safety?**

A: Valid state during errors.

**Q10: throw vs throw;?**

A: New vs re-throw current.

## Topic 20: File I/O

---

File I/O uses `fstream`. `ifstream` reads; `ofstream` writes. `getline` reads lines. Binary uses `read()/write()`. Check `is_open()`.

### Learn From

File I/O references.

- C++ `fstream` — <https://cplusplus.com/doc/tutorial/files/>

### Key Concepts

- `ifstream/ofstream/fstream`
- `open()` and `close()`
- `getline()`
- `is_open()`
- `ios::binary`
- `read()` and `write()`
- File modes

### Problem 1

Write to file.

```
#include <fstream>
int main(){std::ofstream out("test.txt");out<<"Hello File";out.close();return 0;}
```

*Learning points:*

### Problem 2

Read from file.

```
#include <fstream>
#include <iostream>
#include <string>
int main(){std::ifstream in("test.txt");std::string l;std::getline(in,l);std::cout<<l<<std::endl;in.close();return 0;}
```

*Expected output:*

Hello File

*Learning points:*

### Problem 3

Append to file.

```
#include <fstream>
int main(){std::ofstream out("test.txt",std::ios::app);out<<"Append";out.close();return 0;}
```

*Learning points:*

## Problem 4

Read lines.

```
#include <fstream>
#include <iostream>
#include <string>
int main(){std::ofstream out("lines.txt");out<<"Line1\nLine2\nLine3";out.close();std::ifstream in("lines.txt")
```

*Expected output:*

```
Line1
Line2
Line3
```

*Learning points:*

## Problem 5

Check file opened.

```
#include <fstream>
#include <iostream>
int main(){std::ifstream in("noexist.txt");std::cout<<(in.is_open()?"opened":"failed")<<std::endl;return 0;}
```

*Expected output:*

```
failed
```

*Learning points:*

## Problem 6

Write numbers 1-5.

```
#include <fstream>
int main(){std::ofstream out("nums.txt");for(int i=1;i<=5;i++)out<<i<<"\n";out.close();return 0;}
```

*Learning points:*

## Problem 7

Read back numbers.

```
#include <fstream>
#include <iostream>
int main(){std::ifstream in("nums.txt");int n;while(in>>n)std::cout<<n<<std::endl;in.close();return 0;}
```

*Expected output:*

```
1
2
3
4
5
```

*Learning points:*

## Problem 8

Binary write.

```
#include <fstream>
int main(){int d[]={1,2,3};std::ofstream out("bin.dat",std::ios::binary);out.write((char*)d,sizeof(d));out.close();}
```

*Learning points:*

## Problem 9

Binary read.

```
#include <fstream>
#include <iostream>
int main(){int d[3];std::ifstream in("bin.dat",std::ios::binary);in.read((char*)d,sizeof(d));in.close();for(int i=0;i<3;i++)cout<<d[i]<<" ";}
```

*Expected output:*

```
1
2
3
```

*Learning points:*

## Problem 10

Check file size.

```
#include <fstream>
#include <iostream>
int main(){std::ifstream in("bin.dat",std::ios::binary|std::ios::ate);std::cout<<in.tellg()<<std::endl;in.close();}
```

*Expected output:*

```
12
```

*Learning points:*

## Quick Questions & Answers

### Q1: ifstream vs ofstream?

A: Read vs write.

### Q2: getline()?

A: Reads line with spaces.

### Q3: is\_open()?

A: Check if opened.

### Q4: ios::binary?

A: Binary mode.

### Q5: read/write()?

A: Binary I/O.

### Q6: ios::app?

A: Append mode.

### Q7: ios::trunc?

A: Truncate on open.

**Q8: Close when?**

A: When done.

**Q9: tellg()?**

A: Read position.

**Q10: seekg()?**

A: Move read position.



# Topic 21: Preprocessor

---

Preprocessor runs before compilation. `#define` creates macros. `#include` imports headers. Conditional compilation enables platform-specific code.

## Learn From

Preprocessor references.

- C++ Preprocessor — <https://cplusplus.com/doc/tutorial/preprocessor/>

## Key Concepts

- `#define` macros
- `#include` headers
- `#ifdef`/`#ifndef`/`#endif`
- `#pragma` directives
- Macro functions
- Include guards
- `#` and `##` operators

### Problem 1

`#define` constant.

```
#include <iostream>
#define MAX 100
int main(){std::cout<<MAX<<std::endl;return 0;}
```

*Expected output:*

100

*Learning points:*

### Problem 2

`#define` macro function.

```
#include <iostream>
#define SQUARE(x) ((x)*(x))
int main(){std::cout<<SQUARE(5)<<std::endl;return 0;}
```

*Expected output:*

25

*Learning points:*

### Problem 3

`#ifdef` conditional.

```
#include <iostream>
#define DEBUG
int main(){#ifdef DEBUG
std::cout<<"debug"<<std::endl;#endif
return 0;}
```

*Expected output:*

debug

*Learning points:*

## Problem 4

Include guard.

```
#include <iostream>
#ifndef MYHEADER_H
#define MYHEADER_H
int getValue(){return 42;}
#endif
int main(){std::cout<<getValue()<<std::endl;return 0;}
```

*Expected output:*

42

*Learning points:*

## Problem 5

#undef.

```
#include <iostream>
#define TEMP 10
#undef TEMP
int main(){std::cout<<"undef"<<std::endl;return 0;}
```

*Expected output:*

undef

*Learning points:*

## Problem 6

#pragma once.

```
#pragma once
#include <iostream>
int main(){std::cout<<"pragma"<<std::endl;return 0;}
```

*Expected output:*

pragma

*Learning points:*

## Problem 7

Token pasting ##.

```
#include <iostream>
#define MAKE_VAR(x) var_##x
int main(){int MAKE_VAR(1)=10;std::cout<<var_1<<std::endl;return 0;}
```

*Expected output:*

10

*Learning points:*

## Problem 8

Stringize #.

```
#include <iostream>
#define STR(x) #x
int main(){std::cout<<STR>Hello)<<std::endl;return 0;}
```

*Expected output:*

Hello

*Learning points:*

## Problem 9

Platform conditional.

```
#include <iostream>
#ifdef _WIN32
#define PLATFORM "Windows"
#else
#define PLATFORM "Other"
#endif
int main(){std::cout<<PLATFORM<<std::endl;return 0;}
```

*Expected output:*

Other

*Learning points:*

## Problem 10

Nested ifdef.

```
#include <iostream>
#define A
#define B
int main(){#if defined(A)&&defined(B)
std::cout<<"both"<<std::endl;#endif
return 0;}
```

*Expected output:*

both

*Learning points:*

## Quick Questions & Answers

### Q1: #define?

A: Creates macros/constants.

### Q2: Preprocessor?

A: Runs before compilation.

### Q3: #ifdef?

A: Check if defined.

**Q4: #ifndef?**

A: Check if not defined.

**Q5: Include guards?**

A: Prevent multiple inclusion.

**Q6: #pragma?**

A: Compiler-specific directives.

**Q7: ##?**

A: Token pasting.

**Q8: #?**

A: Stringize to string.

**Q9: #undef?**

A: Undefines macro.

**Q10: When preprocessor runs?**

A: Before compilation.

## Topic 22: Memory Layout & Internals

---

Programs have text, data, heap, stack segments. Compilation: preprocess, compile, assemble, link. ELF is Linux; PE is Windows.

### Learn From

Memory references.

- Memory Model — [https://en.cppreference.com/w/cpp/language/memory\\_model](https://en.cppreference.com/w/cpp/language/memory_model)

### Key Concepts

- Text segment
- Data segment
- BSS segment
- Heap
- Stack
- Compilation stages
- ELF and PE formats

### Problem 1

Stack vs heap addresses.

```
#include <iostream>
int main(){int s=1;int *h=new int(2);std::cout<<"stack: "<<&s<<std::endl;std::cout<<"heap: "<<h<<std::endl;de
```

*Expected output:*

stack: 0x

*Learning points:*

### Problem 2

Stack growth direction.

```
#include <iostream>
void func(int *prev){int local;std::cout<<(prev>&local?"down":"up")<<std::endl;}
int main(){int f;func(&f);return 0;}
```

*Expected output:*

down

*Learning points:*

### Problem 3

Segment sizes.

```
#include <iostream>
int gi=1;int gu;
int main(){int l=1;static int si=1;int *h=new int(1);std::cout<<sizeof(l)<<std::endl;delete h;return 0;}
```

*Expected output:*

4

*Learning points:*

## Problem 4

Function address.

```
#include <iostream>
void func(){}
int main(){std::cout<<(void*)func<<std::endl;return 0;}
```

*Expected output:*

0x

*Learning points:*

## Problem 5

Heap allocation speed.

```
#include <iostream>
#include <chrono>
int main(){auto s=std::chrono::high_resolution_clock::now();for(int i=0;i<1000000;i++){int *p=new int(i);delete p;}auto e=std::chrono::high_resolution_clock::now();std::cout<<std::chrono::duration_cast<std::chrono::seconds>(e-s).count()<<std::endl;return 0;}
```

*Learning points:*

## Problem 6

Global vs local addr.

```
#include <iostream>
int gv=10;
int main(){int lv=20;std::cout<<"g: " <<&gv<<std::endl;std::cout<<"l: " <<&lv<<std::endl;return 0;}
```

*Expected output:*

g: 0x

*Learning points:*

## Problem 7

Variable lifetime.

```
#include <iostream>
int main(){std::cout<<"lifetime"<<std::endl;return 0;}
```

*Expected output:*

lifetime

*Learning points:*

## Problem 8

Alignment.

```
#include <iostream>
struct P{char a;int b;char c;};struct A{int a;char b;int c;};
int main(){std::cout<<sizeof(P)<<std::endl;std::cout<<sizeof(A)<<std::endl;return 0;}
```

*Expected output:*

12  
12

*Learning points:*

## Problem 9

Memory regions.

```
#include <iostream>
int g;
int main(){int l;int *h=new int;std::cout<<"code: "<<(void*)main<<std::endl;std::cout<<"data: "<<&g<<std::endl;
```

*Expected output:*

code: 0x

*Learning points:*

## Problem 10

Static storage.

```
#include <iostream>
void counter(){static int c=0;c++;std::cout<<c<<std::endl;}
int main(){counter();counter();counter();return 0;}
```

*Expected output:*

1  
2  
3

*Learning points:*

## Quick Questions & Answers

### Q1: Memory segments?

A: Text, data, BSS, heap, stack.

### Q2: Text?

A: Executable code (read-only).

### Q3: Heap?

A: Dynamic allocation, grows up.

### Q4: Stack?

A: Locals/calls, grows down.

### Q5: ELF?

A: Linux binary format.

### Q6: PE?

A: Windows binary format.

### Q7: Compilation stages?

A: Preprocess, compile, assemble, link.

### Q8: Stack vs heap?

A: Stack fast; heap slow.

**Q9: BSS?**

A: Uninitialized globals.

**Q10: Layout for RE?**

A: Understanding structure and exploitation.



## Topic 23: Bitwise Operations & Manipulation

---

Bitwise operations work on bits. Shifts move bits. Masks isolate bits. Common tricks: even/odd, power of 2, set/clear/toggle.

### Learn From

Bitwise references.

- C++ Bitset — <https://cplusplus.com/reference/bitset/bitset/>

### Key Concepts

- AND & OR | XOR ^ NOT ~
- Left << Right >> shifts
- Bit masks
- Set/clear/toggle
- std::bitset
- Common tricks
- Bit counting

### Problem 1

Set bit 3.

```
#include <iostream>
int main(){int v=0;v|=(1<<3);std::cout<<v<<std::endl;return 0;}
```

*Expected output:*

8

*Learning points:*

### Problem 2

Clear bit 3.

```
#include <iostream>
int main(){int v=0xFF;v&=~(1<<3);std::cout<<v<<std::endl;return 0;}
```

*Expected output:*

247

*Learning points:*

### Problem 3

Toggle bit 0.

```
#include <iostream>
int main(){int v=5;v^=(1<<0);std::cout<<v<<std::endl;return 0;}
```

*Expected output:*

4

*Learning points:*

## Problem 4

Check bit 2 set.

```
#include <iostream>
int main(){int v=5;bool s=(v&(1<<2))!=0;std::cout<<s<<std::endl;return 0;}
```

*Expected output:*

1

*Learning points:*

## Problem 5

Even/odd check.

```
#include <iostream>
int main(){int n=4;std::cout<<(n%2==0?"even":"odd")<<std::endl;return 0;}
```

*Expected output:*

even

*Learning points:*

## Problem 6

Count set bits 7.

```
#include <iostream>
int main(){int n=7;int c=0;while(n){c+=n&1;n>>=1;}std::cout<<c<<std::endl;return 0;}
```

*Expected output:*

3

*Learning points:*

## Problem 7

Bitset binary 10.

```
#include <iostream>
#include <bitset>
int main(){std::cout<<std::bitset<8>(10)<<std::endl;return 0;}
```

*Expected output:*

00001010

*Learning points:*

## Problem 8

XOR swap.

```
#include <iostream>
int main(){int a=3,b=5;a^=b;b^=a;a^=b;std::cout<<a<<" "<<b<<std::endl;return 0;}
```

*Expected output:*

5 3

Learning points:

## Problem 9

Extract byte 1.

```
#include <iostream>
int main(){int v=0x12345678;int b=(v>>8)&0xFF;std::cout<<std::hex<<b<<std::endl;return 0;}
```

Expected output:

56

Learning points:

## Problem 10

Power of 2 check.

```
#include <iostream>
int main(){int n=8;bool p=(n>0)&&((n&(n-1))==0);std::cout<<p<<std::endl;return 0;}
```

Expected output:

1

Learning points:

## Quick Questions & Answers

### Q1: & purpose?

A: AND: clear/check bits.

### Q2: | purpose?

A: OR: set bits.

### Q3: ^ purpose?

A: XOR: toggle, swap.

### Q4: ~ purpose?

A: NOT: flip all bits.

### Q5: << purpose?

A: Left shift:  $\times 2^n$ .

### Q6: >> purpose?

A: Right shift:  $/2^n$ .

### Q7: Bitmask?

A: Value with specific bits.

### Q8: Check bit n?

A:  $(val \& (1 \ll n)) \neq 0$ .

### Q9: Set bit n?

A:  $val |= (1 \ll n)$ .

### Q10: Clear bit n?

A:  $val \&= \sim(1 \ll n)$ .

## Topic 24: WinAPI Basics for RE

---

Windows API essential for RE. Key functions: MessageBox, CreateFile/ReadFile, ReadProcessMemory, PE format structures.

### Learn From

WinAPI references.

- Microsoft WinAPI — <https://docs.microsoft.com/en-us/windows/win32/api/>

### Key Concepts

- MessageBox
- CreateFile and ReadFile
- ReadProcessMemory
- PE format structures
- Virtual Address
- Import/Export tables
- PE loading process

### Problem 1

MessageBox.

```
#include <windows.h>
int WINAPI WinMain(HINSTANCE h,HINSTANCE p,LPSTR cmd,int show){MessageBox(NULL,"Hello RE","Title",MB_OK);return 0;}
```

*Learning points:*

### Problem 2

CreateFile open.

```
#include <windows.h>
#include <iostream>
int main(){HANDLE h=CreateFile("test.txt",GENERIC_READ,0,NULL,OPEN_EXISTING,0,NULL);if(h!=INVALID_HANDLE_VALUE){cout<<"File opened successfully\n";return 0;}else{cout<<"File not found\n";return 1;}}
```

*Expected output:*

opened

*Learning points:*

### Problem 3

ReadFile content.

```
#include <windows.h>
#include <iostream>
int main(){HANDLE h=CreateFile("test.txt",GENERIC_READ,0,NULL,OPEN_EXISTING,0,NULL);char b[256]={0};DWORD r;ReadFile(h,b,256,&r,NULL);cout<<b<<endl;return 0;}
```

*Learning points:*

## Problem 4

GetModuleHandle.

```
#include <windows.h>
#include <iostream>
int main(){HMODULE h=GetModuleHandle(NULL);std::cout<<h<<std::endl;return 0;}
```

*Learning points:*

## Problem 5

GetCurrentProcessId.

```
#include <windows.h>
#include <iostream>
int main(){std::cout<<GetCurrentProcessId()<<std::endl;return 0;}
```

*Learning points:*

## Problem 6

GetLastError.

```
#include <windows.h>
#include <iostream>
int main(){CreateFile("noexist.txt",GENERIC_READ,0,NULL,OPEN_EXISTING,0,NULL);std::cout<<GetLastError()<<std::endl;return 0;}
```

*Expected output:*

2

*Learning points:*

## Problem 7

GetSystemInfo.

```
#include <windows.h>
#include <iostream>
int main(){SYSTEM_INFO si;GetSystemInfo(&si);std::cout<<si.dwNumberOfProcessors<<std::endl;return 0;}
```

*Learning points:*

## Problem 8

VirtualAlloc.

```
#include <windows.h>
#include <iostream>
int main(){void *p=VirtualAlloc(NULL,4096,MEM_COMMIT,PAGE_READWRITE);if(p)std::cout<<"allocated"<<std::endl;return 0;}
```

*Expected output:*

allocated

*Learning points:*

## Problem 9

FindWindow.

```
#include <windows.h>
#include <iostream>
int main(){HWND h=FindWindow(NULL,"Untitled - Notepad");std::cout<<(h?"found":"not found")<<std::endl;return 0}
```

*Expected output:*

not found

*Learning points:*

## Problem 10

GetVersionEx.

```
#include <windows.h>
#include <iostream>
int main(){OSVERSIONINFO oi;oi.dwOSVersionInfoSize=sizeof(oi);GetVersionEx(&oi);std::cout<<oi.dwMajorVersion<<std::endl;return 0}
```

*Expected output:*

6.1

*Learning points:*

## Quick Questions & Answers

### Q1: WinAPI?

A: Windows system calls.

### Q2: CreateFile?

A: Opens/creates files/devices.

### Q3: ReadProcessMemory?

A: Reads another process memory.

### Q4: PE format?

A: Windows binary format.

### Q5: Virtual Address?

A: Process memory address.

### Q6: IMAGE\_NT\_HEADERS?

A: PE header with entry point.

### Q7: Import Table?

A: Lists imported DLLs/functions.

### Q8: Export Table?

A: Lists exported functions.

### Q9: GetModuleHandle?

A: Returns module base address.

### Q10: VA vs RVA?

A: Virtual vs offset from base.

## Topic 25: Capstone Projects

---

Apply all concepts in complete projects. Reinforce fundamentals, OOP, STL, memory management, and system programming.

### Learn From

Practice projects.

- GitHub C++ Projects — <https://github.com/topics/cpp-projects>

### Key Concepts

- Combining all topics
- Architecture and design
- Error handling
- Memory management
- OOP patterns
- STL usage
- File I/O
- System programming

### Problem 1

Contact Book.

```
#include <iostream>
#include <vector>
#include <string>
struct Contact{std::string name,phone;};
class Book{std::vector<Contact> c;public:
void add(std::string n,std::string p){c.push_back({n,p});}
void print(){for(auto &x:c)std::cout<<x.name<<" ":<<x.phone<<std::endl;}};
int main(){Book b;b.add("Alice","123");b.add("Bob","456");b.print();return 0;}
```

*Expected output:*

Alice: 123

Bob: 456

*Learning points:*

### Problem 2

Calculator operator overloading.

```
#include <iostream>
class Calc{public:
double add(double a,double b){return a+b;}
double sub(double a,double b){return a-b;}
double mul(double a,double b){return a*b;}
double div(double a,double b){return b!=0?a/b:0;}};
int main(){Calc c;std::cout<<c.add(10,5)<<std::endl;std::cout<<c.mul(3,4)<<std::endl;return 0;}
```

*Expected output:*

15

12

*Learning points:*

### Problem 3

Matrix class.

```
#include <iostream>
#include <vector>
class Matrix{std::vector<std::vector<int>> d;public:
Matrix(int r,int c):d(r,std::vector<int>(c,0)){};
int main(){Matrix m(3,3);std::cout<<"matrix"<<std::endl;return 0;}
```

*Expected output:*

matrix

*Learning points:*

### Problem 4

Linked list template.

```
#include <iostream>
template<typename T>
class Node{public:T data;Node *next;Node(T d):data(d),next(nullptr){};
int main(){Node<int> *h=new Node<int>(1);h->next=new Node<int>(2);std::cout<<h->data<<" "<<h->next->data<<std::endl;return 0;}
```

*Expected output:*

1 2

*Learning points:*

### Problem 5

Run-length compression.

```
#include <iostream>
#include <string>
std::string compress(const std::string &s){std::string r;int i=0;while(i<(int)s.length()){char c=s[i];int cnt=1;while(i+1<(int)s.length()&& s[i+1]==c){i++;cnt++;}r+=c+std::to_string(cnt);i++;}return r;}
int main(){std::cout<<compress("aaabbbcc")<<std::endl;return 0;}
```

*Expected output:*

a3b3c2

*Learning points:*

### Problem 6

Memory pool allocator.

```
#include <iostream>
#include <vector>
class Pool{std::vector<char> m;public:Pool(int s):m(s){}void *alloc(int s){return (void*)m.data();}};
int main(){Pool p(1024);p.alloc(64);std::cout<<"allocated"<<std::endl;return 0;}
```

*Expected output:*

allocated

*Learning points:*



## Problem 7

Template container.

```
#include <iostream>
#include <vector>
template<typename T>
class MyCont{std::vector<T> d;public:
void push(T v){d.push_back(v);}
auto begin(){return d.begin();}
auto end(){return d.end();}};
int main(){MyCont<int> c;c.push(1);c.push(2);c.push(3);for(auto x:c)std::cout<<x<<std::endl;return 0;}
```

*Expected output:*

```
1
2
3
```

*Learning points:*

## Problem 8

Smart pointer custom deleter.

```
#include <iostream>
#include <memory>
struct R{R(){std::cout<<"acq"<<std::endl;}~R(){std::cout<<"rel"<<std::endl;}};
int main(){std::unique_ptr<R,void(*)> p(new R(),[](R *r){delete r;});return 0;}
```

*Expected output:*

```
acq
rel
```

*Learning points:*

## Problem 9

PE header parser.

```
#include <iostream>
#include <cstdint>
struct PE{uint16_t machine;uint16_t sections;};
int main(){PE h;h.machine=0x14c;h.sections=3;std::cout<<"machine: "<<h.machine<<std::endl;std::cout<<"sections: "<<h.sections<<std::endl;return 0;}
```

*Expected output:*

```
machine: 332
sections: 3
```

*Learning points:*

## Problem 10

Comprehensive review.

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
template<typename T>
T max_val(T a,T b){return a>b?a:b;}
class App{public:
void run(){std::cout<<"All topics covered!"<<std::endl;}};
int main(){App app;app.run();auto p=std::make_unique<int>(42);std::cout<<*p<<std::endl;return 0;}
```

*Expected output:*

All topics covered!  
42

*Learning points:*

## Quick Questions & Answers

### Q1: Architecture?

A: Plan before coding.

### Q2: Error handling?

A: Use try-catch.

### Q3: Memory practice?

A: Use smart pointers.

### Q4: OOP in projects?

A: Encapsulation, inheritance, polymorphism.

### Q5: STL practice?

A: Containers and algorithms.

### Q6: File I/O?

A: Serialize state.

### Q7: Templates?

A: Generic, reusable code.

### Q8: Testing?

A: Verify components.

### Q9: Performance?

A: Profile first.

### Q10: Code organization?

A: Separate concerns.